

Sistema de coordenadas y cálculos del juego

Guía visual para documentar movimiento, captura, robo y victoria

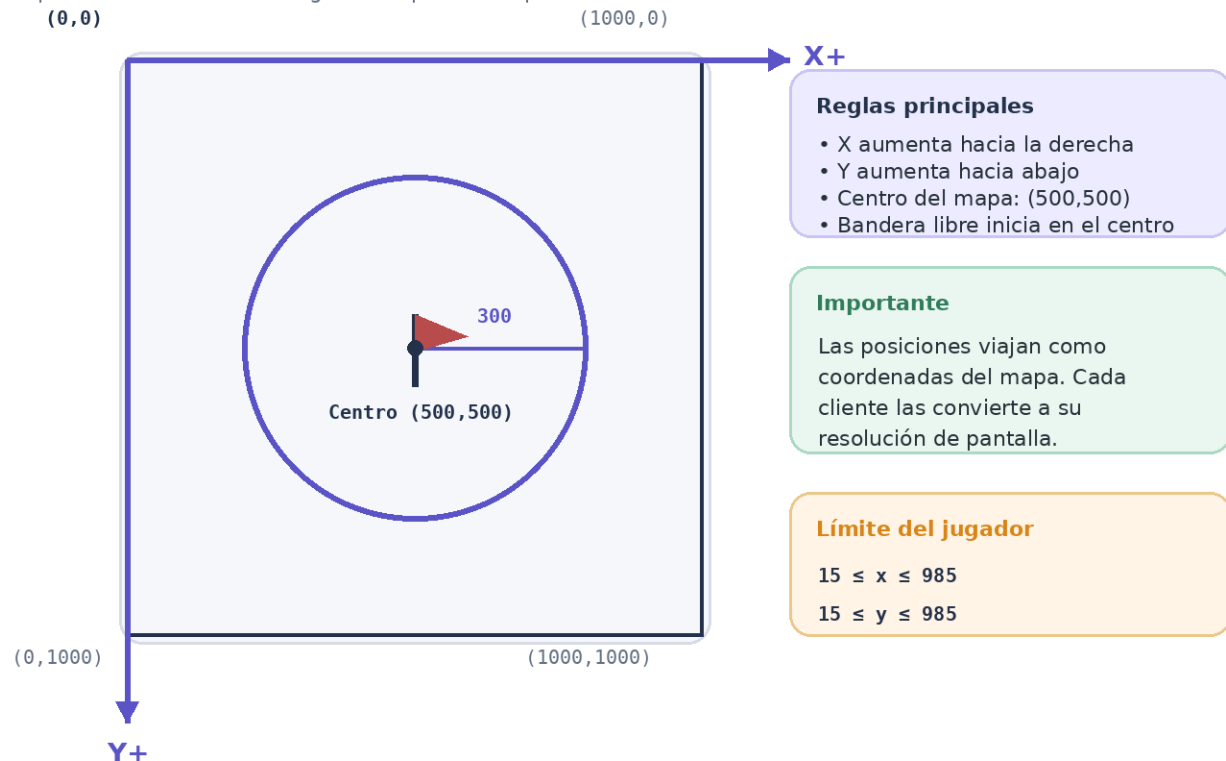
Captura la Bandera · CC8 2026

Objetivo de esta sección

- Definir cómo se representan las posiciones dentro del mapa.
- Establecer cómo el servidor calcula una nueva posición.
- Usar una misma regla para captura, robo y condición de victoria.
- Evitar diferencias entre clientes escritos en distintos lenguajes.

Mapa lógico y sistema de coordenadas

El protocolo usa unidades lógicas, no píxeles de pantalla.



1. Base del sistema de coordenadas

Elemento	Valor	Interpretación
Tamaño del mapa	1000 × 1000	Unidades lógicas, no píxeles
Origen	(0,0)	Esquina superior izquierda
Eje X	Positivo hacia la derecha	Negativo hacia la izquierda
Eje Y	Positivo hacia abajo	Negativo hacia arriba
Centro del mapa	(500,500)	Posición inicial de la bandera libre

Regla de compatibilidad

- Las coordenadas que viajan por la red representan posiciones del mapa.
- Cada cliente convierte esas coordenadas al tamaño de su propia ventana.
- Una pantalla de 800×600 y otra de 1920×1080 deben mostrar el mismo punto lógico.

2. Dirección de movimiento

Las 9 combinaciones posibles de dir (x, y)

El cliente envía intención de movimiento; el servidor calcula la posición.



Regla: y negativo = arriba; y positivo = abajo.

Mensaje de ejemplo: {"type": "input", "dir": {"x": 1, "y": -1}} → arriba y derecha.

Responsabilidad de cada parte

- Cliente: detecta las teclas y envía dir.x y dir.y.
- Servidor: calcula la posición oficial, aplica límites y valida acciones.
- Cliente: dibuja el estado recibido; no decide por sí solo dónde está el jugador.

3. Cálculo de una nueva posición

Para cada actualización se utilizan la posición actual, la dirección, la velocidad y el tiempo transcurrido (dt).

$$m = \sqrt{\text{dir.x}^2 + \text{dir.y}^2}$$

$$\text{normal_x} = \text{dir.x} / m \quad \text{normal_y} = \text{dir.y} / m$$

$$\text{new_x} = \text{current_x} + \text{normal_x} \times \text{speed} \times \text{dt}$$

$$\text{new_y} = \text{current_y} + \text{normal_y} \times \text{speed} \times \text{dt}$$

Cálculo de una nueva posición

Ejemplo: posición (100,200), dirección (1,-1), velocidad 200 y dt = 0.05 s

$$m = \sqrt{\text{dir.x}^2 + \text{dir.y}^2}$$

$$n = (\text{dir.x} / m, \text{dir.y} / m)$$

1

Magnitud

$$m = \sqrt{1^2 + (-1)^2} = 1.414$$

2

Dirección normalizada

$$n = (0.707, -0.707)$$

3

Desplazamiento

$$\Delta = n \times 200 \times 0.05 = (7.07, -7.07)$$

4

Nueva posición

$$(100, 200) + (7.07, -7.07) = (107.07, 192.93)$$

Resultado: el jugador avanza 10 unidades en total, sin ganar velocidad por ir en diagonal.

Caso quieto

- Si $\text{dir.x} = 0$ y $\text{dir.y} = 0$, la magnitud también es 0.
- En ese caso no se divide entre 0: el servidor conserva la posición actual.

¿Por qué se normaliza la diagonal?

Sin normalización, la dirección (1,1) aplicaría velocidad completa en ambos ejes y el jugador avanzaría más rápido. Al normalizar, horizontal, vertical y diagonal conservan la misma rapidez total de **200 unidades por segundo**.

4. Límites del mapa

Como el jugador tiene un radio de 15 unidades, su centro debe permanecer entre 15 y 985 en ambos ejes.

$$\text{min} = \text{player_radius} = 15$$

$$\text{max} = \text{map_size} - \text{player_radius} = 1000 - 15 = 985$$

```
final_x = clamp(new_x, 15, 985)
```

```
final_y = clamp(new_y, 15, 985)
```

Qué hace clamp

- Si el valor es menor que 15, devuelve 15.
- Si el valor es mayor que 985, devuelve 985.
- Si está entre ambos límites, conserva el valor calculado.

5. Distancia entre dos puntos

La misma fórmula sirve para saber si el jugador está cerca de la bandera, de otro jugador o del centro del círculo.

```
distancia = sqrt((x2 - x1)2 + (y2 - y1)2)
```

Dato	Valor
Jugador	(520,510)
Bandera	(500,500)
Resultado	$\text{sqrt}(20^2 + 10^2) = 22.36$
Comparación	$22.36 \leq 40 \rightarrow$ puede interactuar

6. Captura, robo y victoria

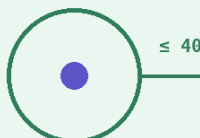
Distancia, captura, robo y victoria

Todas las decisiones se validan en el servidor.

```
distancia = sqrt((x2 - x1)2 + (y2 - y1)2)
```

CAPTURA

- Bandera libre
- Cliente envía interact
- $\text{distancia}(\text{jugador}, \text{bandera}) \leq 40$
- $\rightarrow$ captura aceptada



ROBO

- Bandera en poder de otro jugador
- Cliente envía interact
- $\text{distancia}(\text{ladrón}, \text{portador}) \leq 40$
- $\rightarrow$ cambia el dueño de la bandera



VICTORIA

- Jugador posee la bandera
- Antes: $\text{distancia al centro} \leq 315$
- Después: $\text{distancia al centro} > 315$
- $\rightarrow$ gana al cruzar completamente el borde



$$315 = 300 + 15$$

Regla central: el cliente solicita; el servidor calcula distancia, valida y decide.

Condiciones resumidas

Acción	Condición	Resultado
Captura	bandera libre + interact + distancia ≤ 40	owner = jugador
Robo	bandera cargada + ladrón distinto + interact + distancia ≤ 40	owner = ladrón
Victoria	posee bandera + antes ≤ 315 + después > 315	finaliza la partida

Por qué 315 para la victoria

- El círculo tiene radio 300.
- El jugador tiene radio 15.
- Para quedar completamente afuera: distancia al centro $> 300 + 15 = 315$.
- Si la distancia es exactamente 315, todavía está tocando el borde.

7. Flujo completo que debe implementar el servidor

Flujo completo del movimiento



La posición oficial siempre queda en el servidor.

Reglas finales para copiar al protocolo

Tema	Regla acordada
Coordenadas	Mapa lógico de 1000×1000; origen arriba-izquierda; X derecha; Y abajo.
Entrada	El cliente envía únicamente dir.x y dir.y con valores -1, 0 o 1.
Movimiento	El servidor normaliza la dirección y calcula la nueva posición usando speed y dt.
Velocidad	La rapidez total es 200 unidades/s en cualquier dirección.
Límites	El centro del jugador permanece entre 15 y 985 en ambos ejes.
Captura	Bandera libre + interact + distancia ≤ 40 .
Robo	Bandera cargada + interact + distancia ladrón-portador ≤ 40 .
Victoria	El portador gana al cruzar de distancia ≤ 315 a distancia > 315 .
Autoridad	El servidor valida y decide; los clientes muestran el estado recibido.

Nota de implementación

- El protocolo define las reglas comunes; cada equipo puede programarlas en su lenguaje.
- Los nombres de funciones pueden cambiar, pero los resultados y las condiciones deben ser iguales.
- Los mensajes de posición y dirección deben usar números JSON, no texto.

